Scalable Hardware Pruning through Semiformal Verification and Microarchitecture Awareness

ABSTRACT

As we approach the practical limits of Moore's Law and move beyond Dennard scaling, Application Specific Integrated Processor (ASIP) has attracted increasing interest due to its potential for trading generality for reduced area, improved performance, and lower power consumption. Efficient and effective pruning of unused hardware under application-specific constraints constitutes a fundamental challenge in ASIP design. While existing approaches have made significant progress, they remain fundamentally limited by attempting complete state space exploration while maintaining microarchitecture independence—the former causing severe scalability bottlenecks and the latter sacrificing substantial optimization opportunities. In this work, we propose an agile versatile hardware pruning framework for ASIP design that embraces both semiformal analysis and microarchitecture awareness. First, we employ bounded model checking and k-step induction, trading proof completeness for practical scalability—enabling analysis of industrial-scale designs that would be intractable under unbounded verification. Second, we translate microarchitecture-aware constraints—including ISA, data, and timing-level constraints—into External Don't Care (EDC) and Observability Don't Care (ODC), exposing optimization opportunities invisible to architecture-agnostic methods. Third, we define a Constraint-based RISC-V Instruction Specification Protocol (CRISP) using denotational semantics to facilitate the expression and SystemVerilog assertion (SVA) generation of all aforementioned constraints.

Under identical constraints, our framework produces implementations that are up to 49.6% smaller than state-of-the-art additive ASIP design methodology (RISSP). Relative to prior hardware-trimming frameworks, it achieves 3–4 orders of magnitude faster analysis runtime and delivers 1.96× greater area reduction on the same constrained multiplier benchmark. A further RSD out-of-order case study shows how locally proved invariants guide automatic component pruning. Separately proved source cuts in scheduler classification and load/store dependence logic reduce whole-core cell area by 3.85% at unchanged capacities under explicit execution contracts.

KEYWORDS

Formal method, ASIP, Observability Don't Care

1 INTRODUCTION

Resource-constrained computing platforms continue to expand across diverse application domains, from IoT edge devices to implantable biomedical systems to flexible electronics for wearable computing. These systems face stringent constraints on silicon area, power budgets, and manufacturing costs that make conventional processor architectures increasingly impractical. Flexible and printed electronics exemplify this challenge: with transistor counts measured in thousands rather than billions and mask costs approaching zero, the economics fundamentally favor highly specialized designs over general-purpose solutions [3, 13]. ASIPs offer a promising approach to these challenges by customizing processor architectures to specific workload requirements, enabling dramatic reductions in area and power consumption compared to general-purpose designs.

Traditional ASIP methodologies primarily focus on enhancing performance through instruction set extensions, adding specialized operations to accelerate critical computations. However, this additive strategy contradicts the fundamental need for smaller, more efficient processors in resource-constrained environments. Adding custom instructions increases rather than decreases silicon footprint, exacerbating the very problem these systems need to solve. Recent analysis of production workloads reveals a compelling alternative: typical applications exercise only a fraction of available processor functionality [4, 15], with large portions of hardware remaining dormant throughout execution. This observation motivates a subtractive approach to specialization, eliminating unused capabilities rather than adding new ones.

Recognition of this opportunity has spurred significant research into hardware pruning techniques for processor optimization. Pioneering work on Bespoke microprocessors [6] established the feasibility of automatically tailoring processors to individual applications by removing untoggled gates. Subsequent efforts expanded this concept: PDAG [4] introduced formal verification methods to ensure correctness while supporting diverse ISAs and microarchitectures, and FSYN [10] provided open-source implementations of these techniques; RISC-V Instruction Subset Processors (RISSP) [15] focused specifically on RISC-V implementations with simplified microarchitectures. Additional research directions include approximate behavioral processors [18] that exploit error tolerance for further optimization, enhanced symbolic execution methods [17] for improved analysis coverage, accelerator subsetting [20] that extends these concepts beyond processors, and formal profiling techniques [19] for systematic workload characterization.

However, these existing approaches share fundamental limitations that constrain their practical applicability and optimization potential. First, subtractive processes are inherently conservative: they must formally prove that hardware structures are unnecessary, requiring expensive symbolic execution or model checking that often becomes intractable for industrial-scale designs. RISSP addresses this scalability challenge by adopting a hardware-additive approach—starting from a base architecture with no functional units and adding them as necessary—thereby avoiding the need for expensive proofs of unnecessity [15]. Second, most approaches remain microarchitecture-unaware, considering only EDCs at the ISA

level. This architectural myopia prevents them from exploiting data-level and interface-level constraints, and critically, causes them to overlook substantial ODCs that emerge from the interaction between ISA-level controllability constraints and microarchitectural implementation details. Third, while RISSP achieves both scalability and microarchitecture awareness, its hardware-additive approach requires committing to a specific microarchitectural template, sacrificing the generality required for diverse design contexts—a limitation analogous to ISA-additive approaches. The open-source FSYN is only demonstrated on processor subcircuits (e.g., controller, multiplier), and scales poorly up to full microprocessors.

What is needed is a hardware pruning framework that addresses all three limitations simultaneously. Such a tool must satisfy four critical requirements: (1) open-source availability to minimize non-recurring engineering costs—particularly important in domains like flexible and printed electronics where EDA licensing costs can exceed mask and equipment costs, (2) generality across microcontroller implementations without requiring a fixed hardware template, (3) support for expressing non-ISA constraints including data-level and interface-level specifications, and (4) microarchitecture-aware optimization capabilities that enable targeted exploitation of ODCs with acceptable computational overhead.

In this work, we propose Property-Driven Analysis and Generation via Semiformal Analysis (PDAG-SFA), a hardware pruning framework for ASIP design that combines property-driven analysis with signal correspondence to achieve both semiformal verification and microarchitecture awareness. Our approach makes three key contributions. First, we employ semiformal techniques to automated hardware removal, which enables practical scalability and speed, trading proof completeness for the ability to analyze industrial-scale designs that would be intractable or time consuming under unbounded verification. Second, we translate microarchitecture-aware constraints—including ISA-level, data-level, and interface-level specifications—into EDCs and ODCs, exposing optimization opportunities that remain invisible to microarchitecture-agnostic methods. Third, we define a domain-specific language (CRISP) using denotational semantics to facilitate precise expression and integration of all aforementioned constraints.

Through these contributions, PDAG-SFA delivers up to 49.6% area reduction compared with the state-of-the-art additive ASIP design approach. Relative to prior trimming frameworks, it achieves 3–4 orders of magnitude faster analysis and provides up to 1.96× greater area reduction on the same benchmark under the same constraints.

An additional out-of-order RSD case study examines specialization inside required scheduler and load/store structures while retaining dynamic scheduling (Section 4.1). It distinguishes these cuts from resizing the ROB, issue queue, and physical registers. It also exposes a constraint boundary absent from committed-instruction reasoning: speculative execution with unavailable operands can violate an otherwise valid application-level address partition.

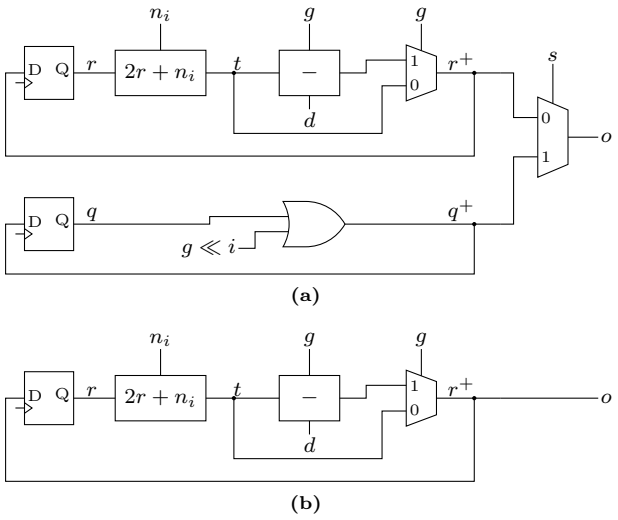


Figure 1: Normalized divider datapaths before (a) and after (b) quotient-cone removal. The subtract decision is $g = [t \geq d]$; $s = 0$ selects remainder and $s = 1$ quotient. Retaining only `REM/REMU` fixes $s = 0$, making the quotient feedback unobservable. Sign handling, initialization, and completion control are omitted and preserved.

2 EFFICIENTLY EXPLOITING EDCs FOR LOGIC OPTIMIZATION

2.1 Motivating Example: A Divider Without Quotient State

A processor that retains `REM/REMU` but excludes `DIV/DIVU` still needs its divider. Yet the divider may compute and store a quotient throughout every remainder operation. Neither removing unused functional units nor testing quotient state for constancy exposes this redundancy: *active state inside a required unit can be irrelevant to every permitted observation.*

Ibex's shared multiply/divide unit [16][1] accumulates quotient bits in `op_quotient_q` during both division and remainder operations. The signal `next_quotient` feeds this recurrence and, in the final arithmetic state, supplies the result only for division. Figure 1(a) expresses the unsigned arithmetic core in normalized form. For numerator bit n_i, divisor d, partial remainder r, and quotient q, each round computes

$$t = 2r + n_i, \qquad g = [t \geq d],$$
$$r^+ = g \ ? \ (t - d) : t, \qquad q^+ = q \vee (g \ll i). \tag{1}$$

The counter visits bit positions 31 through 0. The decision g is shared: it selects the new remainder and sets a quotient bit. For 13 mod 3, the last four rounds produce remainders $1, 0, 0, 1$ and quotients $0, 4, 4, 4$. The quotient changes during a supported instruction, although only the remainder is needed.

[1] Ibex RTL, revision `1f2232a`, lines 390–519.

Why the safe cut is subtle. Removing the divider would destroy remainder instructions. Removing the subtract decision or counter would also be wrong: both feed the remainder recurrence. The safe target is the *quotient-exclusive feedback cone*. Microarchitecture information locates its boundary at the internal result selection. Injecting an arbitrary error at `next_quotient` tests whether that error can escape into a permitted result [8]. After this check succeeds, tying the signal to zero makes the quotient register and its bit-insertion logic removable, while preserving the shared arithmetic.

A sequential proof with a smaller state relation. The decoded operation must remain associated with each accepted remainder transaction until completion. Under this contract, the quotient result branch is never selected. Moreover, q feeds neither the remainder recurrence nor operand preparation, remainder sign correction, the counter, or completion control. Relate the two designs by equality of these retained states, allowing quotient state to differ. Reset establishes the relation; Eq. (1) preserves it on each enabled round, and a stall holds it. Result backpressure also preserves the common result and valid signal. The circuits therefore return identical remainders on identical cycles despite different quotient histories. This combines reachability and observability [12] without requiring equality of irrelevant internal state.

The specialization removes the 32-bit quotient register and its exclusive update logic, including 31 potentially non-constant state bits (Fig. 1(b)). We checked the state relation in an independent inductive model and compared the original Ibex RTL with a quotient-path cut on 264 signed and unsigned remainder transactions, including zero divisors and output backpressure. This validates the example, not automatic candidate discovery or mapped-area savings. It motivates PDAG-SFA's combination of microarchitecture-guided candidate selection and constraint-aware sequential analysis: pruning must reach inside required units and reason about the observations of stateful subcircuits.

2.2 Property Driven Generation of Reduced ISA Hardware via Semiformal Analysis

PDAG-SFA targets candidate removals and checks them under the supplied constraints (Fig. 2). It takes three inputs: the **RTL design**, **CRISP constraints** specifying the supported instruction subset (Section 3), and a core-specific **configuration**. The configuration maps clock, reset, and instruction signals; identifies constraint injection sites, result-selection paths, and the architectural observation boundary; and supplies an initial induction depth. It also supports data-level and interface-level constraints. These inputs guide candidate selection and define the assumptions under which each removal must preserve behavior.

2.2.1 Synthesis with Constraints. These three inputs are synthesized into an And-Inverter Graph (AIG) representation with CRISP constraints embedded as assumptions using the open-source synthesizer Synlig [7]. The RTL is first elaborated and the microarchitecture configuration specifies where constraint assertions should be injected into the design hierarchy. The resulting AIG encodes both the circuit structure and the behavioral constraints under which optimization must preserve correctness.

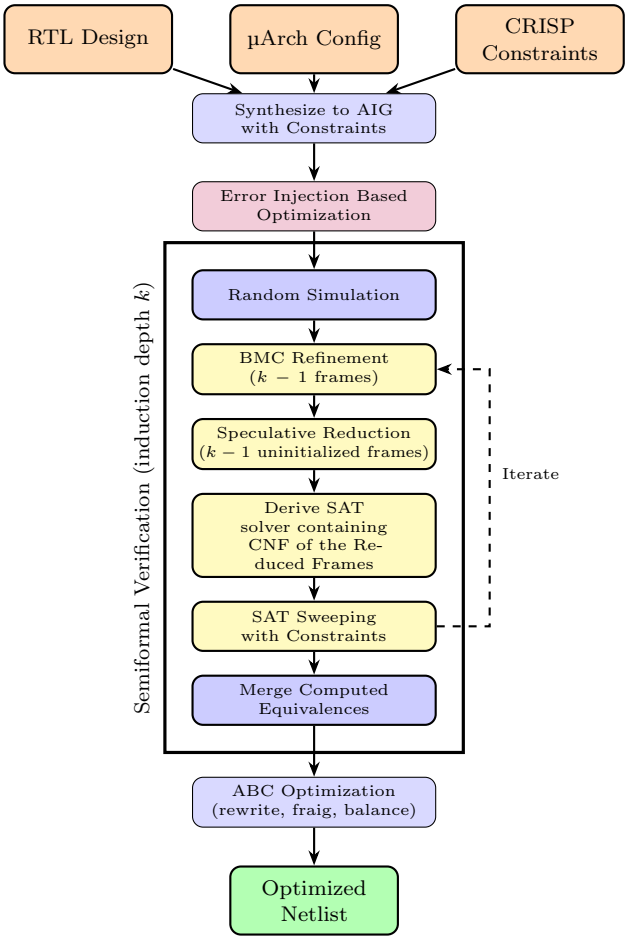


Figure 2: PDAG-SFA workflow: RTL design and CRISP constraints are synthesized to an AIG with embedded assumptions. Microarchitecture-aware error injection identifies candidate optimization targets, which are checked through SAT-based sequential equivalence using k-induction, with pipeline depth guiding the initial choice of k. Iterative refinement via random simulation, BMC, and speculative reduction enables scalable verification. Proven equivalences are merged and further optimized using ABC's algebraic rewriting and technology-independent optimization passes [5, 12, 14].

2.2.2 Microarchitecture-Aware Error Injection. Rather than exhaustively injecting errors at every signal in the design, PDAG-SFA leverages the microarchitecture configuration and CRISP constraints to identify promising optimization candidates. By analyzing which instructions are forbidden and examining the cores' multiplexers' structures, the tool

determines which functional unit outputs are likely unobservable. For example, if CRISP constraints forbid all division instructions, the microarchitecture configuration identifies the divider output in the ALU result mux, enabling targeted error injection at this signal. This microarchitecture-aware targeting dramatically reduces the verification problem space compared to naïve enumeration of all possible error injection sites, while maintaining soundness.

A sound removal certificate must establish that the injected error cannot affect the configured architectural boundary, including register-file updates, CSRs, and memory-interface effects. The motivating example discharges this obligation with an inductive invariant. Internal state remains in the transition relation even when its equality is not an observation requirement. SAT-based sequential equivalence can discharge this obligation with k-induction: the base case establishes equivalence from reset, and the inductive step establishes its preservation. Pipeline depth supplies an initial choice of k, not a completeness bound; stalls and feedback may require deeper induction or auxiliary invariants. Random simulation and BMC refine candidate correspondences, while speculative reduction simplifies the proof problems. Only candidates with successful base and inductive checks justify removal. A timeout or an inconclusive check provides no removal certificate. Candidates accepted solely on BMC evidence have only a bounded guarantee and fall outside this inductive proof contract. Thus, bounded unrolling controls the size of each SAT query, while induction provides an unbounded guarantee for the transformations it proves under the stated constraints.

2.2.3 Sequential SAT Sweeping. After error injection-based optimization, PDAG-SFA applies sequential SAT sweeping [14] as a complementary, orthogonal optimization technique. Sequential SAT sweeping identifies functionally equivalent signals within the circuit by first simulating with random inputs and hashing signal value sequences to find potentially equivalent net pairs. Candidate equivalences are then checked via k-induction: both the base case (equivalence for the first $k-1$ cycles) and the inductive step (equivalence in cycle k follows from equivalence in the previous $k-1$ cycles) reduce to bounded Sequential Equivalence Checking (SEC) problems solved via SAT. Critically, these equivalence checks are performed under the CRISP constraints embedded as assumptions in the AIG. The SAT solver only needs to consider behaviors permitted by the ISA-level, data-level, and interface-level constraints, enabling proofs of equivalence that would be invalid in the unconstrained circuit. The combination of microarchitecture-aware error injection and constraint-aware sequential SAT sweeping yields a three orders of magnitude improvement in runtime compared to unbounded verification approaches (Table 2).

2.2.4 Optimization and Output. Proven equivalences are merged into the AIG, eliminating signals shown to be unobservable under the given constraints. The optimized AIG then undergoes further technology-independent optimization using ABC's algebraic rewriting (`rewrite`), functional reduction via AIGs (`fraig`), and structural balancing (`balance`) passes [5]. The final optimized AIG can be mapped to standard cells for physical implementation.

Table 1: Core area (Skywater 130 nm) of a single-cycle RV32E microprocessor under various ISA subset assumptions. A ✓ indicates the presence of instructions using the hardware multiplier, divider, adder, and barrel shifter, while a ✗ indicates the absence of those instructions. Cores generated by PDAG-SFA are, without exception, smaller than those generated through RISSP's additive processing.

Mul	Div	Add	Shift	RISSP (μm^2)	PDAG-SFA (μm^2)	%Δ
✗	✗	✗	✗	33148.04	32205.90	-2.84%
✗	✗	✗	✓	36862.85	35661.70	-3.26%
✗	✗	✓	✗	35940.72	32205.90	-10.39%
✗	✗	✓	✓	39303.95	35609.20	-9.40%
✗	✓	✗	✗	60857.12	31996.90	-47.42%
✗	✓	✗	✓	64291.66	35521.60	-44.75%
✗	✓	✓	✗	63397.05	31950.60	-49.60%
✗	✓	✓	✓	66939.20	35376.40	-47.15%
✓	✗	✗	✗	85254.27	77455.50	-9.15%
✓	✗	✗	✓	82890.75	81249.20	-1.98%
✓	✗	✓	✗	85784.77	77583.20	-9.56%
✓	✗	✓	✓	88697.57	80835.00	-8.86%
✓	✓	✗	✗	81197.88	77586.90	-4.45%
✓	✓	✗	✓	83980.54	80808.80	-3.78%
✓	✓	✓	✗	100503.89	77327.90	-23.06%
✓	✓	✓	✓	104164.90	80835.00	-22.40%

Table 1 demonstrates that PDAG-SFA's gate-level analysis consistently captures equal or greater area reductions compared to RISSP's module-level additive approach across all ISA subset configurations. While both methods remove entire functional units when their instructions are forbidden (e.g., removing the divider when division instructions are unused), PDAG-SFA achieves 44–50% greater reduction in these cases by identifying additional optimization opportunities in control logic, multiplexing networks, and glue logic surrounding the removed modules. More significantly, even when all functional units are required (final row: all modules present), PDAG-SFA still achieves 22% area reduction—a capability entirely beyond RISSP's reach. This phenomenon, first identified in the original Property-Driven Analysis and Generation (PDAG) work [4], arises because gate-level analysis exploits ODCs within functional units themselves. Even when a module is necessary, ISA constraints render portions of its internal logic unobservable: a multiplier may contain special-case handling for zero operands, sign extension logic for narrower data types, or overflow detection circuits that become redundant under specific instruction subset constraints. RISSP's module-level approach must include or exclude entire functional units as monolithic blocks, while PDAG-SFA's AIG-level signal correspondence analysis systematically identifies and eliminates unobservable gates within required modules.

We compared PDAG-SFA against FSYN [10] using a GenMul-generated 32×32 multiplier with randomly generated

Table 2: Runtime for PDAG-SFA and PDAG on 2-stage version of Ibex [16]. PDAG analysis was performed with JasperGold with timeout set for five core hours.

ISA Subset	PDAG-SFA	Completed	PDAG	Completed
RV32IM	12 s	✓	> 5 h	✗
RV32I	12 s	✓	> 5 h	✗

external constraints (4 input bits fixed to zero), following [10]. Table 3 compares PDAG-SFA against both FSYN variants: without filtering and with the optional rule filtering phase (40 stimuli). On this benchmark, PDAG-SFA achieves 26.69% gate reduction versus FSYN's 13.88% while completing in 7 seconds versus 2.5 hours. This suggests that semiformal signal correspondence can be both significantly faster and more effective than formal verification for constraint-based optimization.

We attempted additional evaluation on EPFL benchmarks [1], including mem_ctrl and arbiter; however, the constraint configurations used in [10] for these circuits were not specified. Our trials with randomly generated constraints resulted in runtimes exceeding 5 hours or zero optimization, indicating that further investigation of appropriate constraint selection for these designs is needed.

Table 3: Gate Reduction Comparison: PDAG-SFA vs FSYN on GenMul 32×32 Multiplier

Metric	FSYN (no filter)	FSYN (filter-40)	PDAG-SFA
Initial size (gates)	11,563	11,563	11,967
Optimized size (gates)	9,958	9,986	8,773
Gates removed	1,605	1,577	3,194
Reduction (%)	**13.88**	**13.64**	**26.69**
Runtime (h:mm:ss)	**2:28:57**	**2:15:55**	**0:00:07**

3 CRISP

In order to effectively express application specific constraints and automatically generate corresponding SystemVerilog assertion, we use a domain-specific language to imperatively add and remove ISA subsets from an (initially empty) allowed subset. Prior works force users to express constraints by manually construct SVA, which can be both tedious and buggy.

To support this, the DSL supports two statements:

Let I be the set of all RISC-V instructions. The 'state' of the language at any point of processing is a set of allowed instructions, which we denote $S_{current} \subseteq I$. The semantic domain for a CRISP program (or, sequence of statements) is a function that maps an initial set S_0 to a final state, S_f.

Let $\mathscr{E}$ be the semantic function mapping expressions in CRISP to RISC-V instructions, i.e., $\mathscr{E} : \text{Expr} \to \mathscr{P}(I)$. Let $\mathscr{S}$ be the semantic function mapping statements to a state transformer function, i.e., $\mathscr{S} : \text{Stmt} \to (\mathcal{P}(I) \to \mathcal{P}(I))$. Let $\mathscr{P}$ be the semantic function mapping programs to a state-transition function: $\mathscr{P} : P \to (\mathcal{P}(I) \to \mathcal{P}(I))$. The meaning of program p is given by $(\mathscr{P}(p))\,(\varnothing)$.

3.1 Expression Syntax and Semantics

CRISP expressions denote sets of RISC-V instructions with five forms:

$$E ::= \text{ext} \mid \text{instr} \mid \text{instr}\{\mathcal{C}\} \mid r_1\text{-}r_2$$

where extensions (e.g., `RV32I`) expand to instruction sets, individual instructions (e.g., `ADD`) to singletons, and constrained instructions apply field restrictions. Field constraints $\mathcal{C} = \{f = v \mid f \texttt{ in } r_1\text{-}r_2\}$ specify equality or range restrictions for fields $f \in \{\texttt{rd}, \texttt{rs1}, \texttt{rs2}, \texttt{imm}\}$.

The semantic function $\mathscr{E}$ maps expressions to instruction sets where each instruction $\iota \in I$ is encoded as (name, pattern, mask, fields):

$$\mathscr{E}[[\text{ext}]] = \{\iota \in I \mid \iota.\text{name} \in \text{instructions(ext)}\} \quad (2)$$

$$\mathscr{E}[[\text{instr}\{\mathcal{C}\}]] = \{\text{constrain}(\iota, \mathcal{C}) \mid \iota \in \mathscr{E}[[\text{instr}]]\} \quad (3)$$

The constraint function refines encodings by updating pattern/mask bits: fixing bits for equality constraints, encoding common prefixes for ranges.

3.2 Statement Syntax and Semantics

CRISP statements transform instruction sets through two operations:

$$S ::= \text{include } E \mid \text{forbid } E \quad (4)$$

with semantics:

$$\mathscr{S}[[\texttt{include } E]] = \lambda s \,.\, s \cup \mathscr{E}[[E]] \quad (5)$$

$$\mathscr{S}[[\texttt{forbid } E]] = \lambda s \,.\, s \setminus \mathscr{E}[[E]] \quad (6)$$

Programs are statement sequences with semantics defined by function composition:

$$\mathscr{P}[[]] = \lambda s \,.\, s \quad (7)$$

$$\mathscr{P}[[\text{Stmt}_1; P_{rest}]] = \mathscr{P}[[P_{rest}]] \circ \mathscr{S}[[\text{Stmt}_1]] \quad (8)$$

The meaning of program p is $\mathscr{P}[[p]](\varnothing)$, processing statements left-to-right from an empty set. For example, `include RV32I, forbid x16-x31` first adds all RV32I instructions, then removes those using registers x16-x31.

3.3 Implementation

CRISP is implemented in Python, representing simple constraints via ranges or enumerations on instruction fields, and complex cross-field constraints via AIGs. The final expression converts to SystemVerilog assumptions injected into RTL/netlists.

3.4 µArch Config File

The configuration YAML serves three critical functions: (1) specifies microarchitectural signals (clock, reset, instruction decoder) and injection points for SVA assertions, (2) defines pipeline depth for k-induction tuning and result multiplexer structure for observability analysis, and (3) enables non-ISA

constraints including data-width restrictions (e.g., limiting operations to 8/16-bit data) and timing-level constraints (e.g., fixed cache latencies) that expose optimization opportunities invisible to ISA-only approaches. Retargeting requires a core-specific signal map, constraint injection sites, and a synthesis wrapper that defines the observable boundary.

4 EXPERIMENTAL RESULTS

We evaluate PDAG-SFA on the Ibex RISC-V core, an open-source 32-bit microcontroller implementing the RV32IMC instruction set. Our experiments demonstrate the framework's ability to optimize hardware under three distinct constraint categories: architectural register constraints, ISA subset constraints, and data-width constraints. All synthesis results target the SkyWater 130 nm open-source PDK using standard optimization settings. We complement the Ibex constraint studies with a Rocket ISA-specialization study to examine portability to a deeper pipeline and the associated analysis cost.

Experimental Setup. We apply PDAG-SFA to generate specialized Ibex variants under different application-specific constraints expressed in our CRISP. Each configuration undergoes the complete optimization flow: constraint specification, semiformal verification to identify unused hardware, and automated removal of unneeded logic. The resulting netlists undergo technology mapping using ABC and static timing analysis with OpenSTA to evaluate area reduction and timing impact. We measure two key metrics: core area and maximum operating frequency (FMax).

Register File Optimization. Figure 3(a) presents the impact of constraining the number of architectural registers from the standard 32 down to 4 registers. This optimization targets embedded applications with limited register pressure, such as simple control loops and state machines. The results demonstrate near-linear area scaling: reducing to 4 registers achieves a 47.3 % area reduction (from 89 873.7 µm^2 to 47 314.1 µm^2) while improving FMax by 80.1 % (from 26.90 MHz to 48.45 MHz). The frequency improvement arises from reduced multiplexer complexity and shorter critical paths in the register file decode logic.

ISA Subset Optimization. Figure 3(b) illustrates the area and performance impact of removing different instruction categories from the base ISA. Starting from the full RV32IM configuration (126 176.0 µm^2), we evaluate progressively restricted instruction sets. Removing the multiply/divide instructions (RV32I) reduces area by 28.7 % to 89 873.7 µm^2 while improving FMax from 22.0 MHz to 26.9 MHz. Further restrictions yield diminishing returns: removing shift instructions saves only an additional 2.7 % area, while an arithmetic-only core (supporting only ADD, SUB, and basic ALU operations) achieves 34.7 % total area reduction compared to RV32IM. Notably, removing branches has minimal impact (0.2 % area reduction), as branch logic shares substantial resources with the base control flow infrastructure that must remain.

Data-Width Optimization. Figure 3(c) demonstrates PDAG-SFA's unique ability to optimize based on data-level constraints that cannot be expressed as ISA restrictions. While RV32IM natively supports only 32-bit operations, many embedded applications operate on smaller data types. By analyzing the actual data widths used in target applications, we can eliminate unused portions of the datapath. Constraining operations to 8-bit data reduces area by 47.6 % (from 89 851.2 µm^2 to 47 125.2 µm^2) and improves FMax by 63.5 %. The 16-bit configuration achieves a balanced trade-off with 31.0 % area reduction while maintaining reasonable performance. These optimizations are particularly valuable for sensor processing and control applications where data naturally fits in smaller widths.

Timing Level Constraint. Table 4 demonstrates the impact of timing-level constraints on the Ibex core area. Ibex employs a request-grant-valid protocol for its data cache (dcache) and instruction cache (icache) interfaces. By constraining the behavior of dcache/icache accesses, the hardware required for protocol implementation can be simplified. Specifically, we constrain the cache access latency such that hits complete in 1 cycle, while misses are guaranteed to resolve within 4 cycles and 5 cycles for dcache and icache, respectively. With timing-level constraints, PDAG-SFA achieves an additional 0.19% area reduction. The improvement is minimal due to the simplicity of Ibex's memory subsystem. However, this demonstrates the capability of applying system-specific constraints at the interface level. For microarchitectures with complex memory systems, timing-level constraints may yield more substantial area reductions. Additionally, such constraints can be used to help resolve timing side channel [11]. For example, it can help expand the set of instructions suitable for fixed-latency programming, effectively enlarging the 'safe' ISA subset [9]. Note that it is the user's responsibility to determine which cache behaviors are appropriate for their specific system requirements.

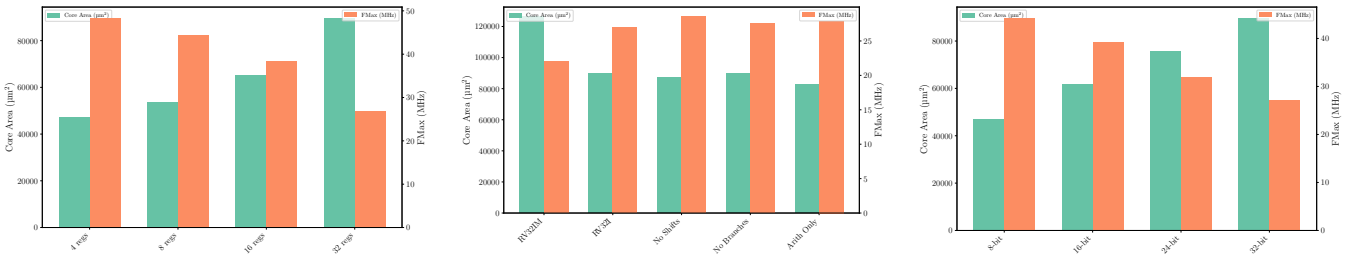
Table 4: Area Comparison Showing Impact of Timing-Level Constraints

Configuration	Area (µm²)		Area Reduction (%)
	Without Timing Level Constraint	With Timing Level Constraint	
RV32IM	126,860.42	126,622.69	0.19
RV32IM_no_mul	99,206.40	99,075.02	0.13
RV32I	89,851.17	89,706.04	0.16

4.1 Microarchitecture-Aware Specialization of an Out-of-Order Core

Out-of-order execution makes a structure necessary without making every field or control path inside it necessary. We study this distinction on RSD, an open-source RV32 core with speculative execution and a nonblocking data cache.[2] The reference retains a two-wide front end, 64-entry reorder

[2]RSD RTL, revision `7b65f6b`.



(a) Register file optimization **(b) ISA subset optimization** **(c) Data-width optimization**

Figure 3: Ibex core optimization results under different constraint categories. (a) Constraining architectural registers from 32 to 4 achieves 47.3% area reduction. (b) ISA subsetting from RV32IM to arithmetic-only achieves 34.7% area reduction. (c) Data-width constraints (impossible to express as ISA constraints) achieve 47.6% area reduction for 8-bit operations. All configurations show consistent FMax improvements with area reduction.

buffer (ROB), 16-entry issue queue (IQ), 64 integer and 64 floating-point (FP) physical registers, 16-entry load and store queues, and two miss-status holding registers (MSHRs). RSD implements reservation-station functionality through a shared IQ, typed payload banks, a producer-matrix wakeup network, and selection logic. We distinguish *pruning at unchanged capacity* from *capacity tuning* across these structures.

Scheduler classification. Our first contract excludes the FP execution class from every actual scheduler write, including wrong-path dispatches. It permits integer, multiply/divide, load, and store classes. Reset-base and temporal-induction checks prove 35 assertions on the original Scheduler and SelectLogic RTL: live entries cannot belong to the excluded class, and FP requests, selections, and grants remain inactive. A cover reaches simultaneously pending work in all four permitted classes; removing the contract produces an FP-request counterexample. A generated candidate ties only the proved FP-request vector to false, preserving IQ capacity and all other classes. This makes 32 classification bits and associated selection logic unused.

The contract concerns micro-operations, not every instruction using an FP register. In RSD, `FLW`/`FSW` use the memory class. Three additional workloads copy raw words through FP registers, including NaN payloads, and still satisfy the scheduler contract. They preserve exact cycles and monitored traces with the cut; independent Spike checks and final FP-register values confirm the copies. Thus this result does not justify removing the FP register file or all FP support.

Historical memory dependence. The second contract requires physical address bits $[21 : 16]$ to be zero for each executed load and nonzero for each executed store. These bits include memory-map attributes. The restriction covers speculative, replayed, and invalid-operand execution. Under it, finished queue entries retain disjoint address regions. Reset-base and temporal induction discharge 36 assertions on the original load/store queue RTL, with a cover reaching simultaneous load/store execution in nonempty queues. A separate

Table 5: Automatic component correspondence: final AND-node counts under the same reset-masked observation boundary. Original RTL request and comparison expressions are retained.

Constraints	Scheduler	LQ+SQ
None	2,847	18,145
Execution/dispatch contract	2,847	17,957
Contract + proved invariants	2,767	13,795

source-audited consumer relation establishes that inactive forwarded data and miss status are unobserved. The candidate cuts only historical address-match vectors; allocation, commitment, recovery, capacities, and same-cycle conflict checking remain. Both candidates are manually identified and locally proved. Neither proof is a full-core equivalence certificate or an all-input program-contract proof.

Guiding automatic correspondence. We also retain the original request/comparison logic and feed the proved entry invariants to the sequential-correspondence backend. Table 5 compares identical component boundaries that observe outputs only when reset is inactive; reset transitions remain present. The controls retain arbitrary initial state, while the added invariants are justified for reset-reachable executions satisfying the event contracts. Their identification and proof are explicit inputs, not automatic invariant discovery. With induction depth 2, a 30,000-conflict per-node limit, and 20 additional counterexample-simulation frames, the invariants enable a further 2.8% and 23.2% AND-node reduction for the scheduler and queue pair, respectively, beyond event assumptions alone. The queue model also loses 112 latches. Retaining reset-time observations instead yields no additional queue reduction from those same invariants. This controlled result exposes the importance of both live-state facts and the observation protocol. These are formal AIG counts, including auxiliary initial-state encoding; they are not physical cell areas.

Table 6: Direct whole-core cell mapping of locally proved source cuts. Each cut requires its stated execution contract.

Source cut	Area (mm^2)	Reduction
None	9.996	—
Load/store queues	9.952	0.44%
Scheduler	9.660	3.36%
Both	9.611	3.85%

Whole-core correspondence remains a scaling limit in this experiment. The baseline export contains 2.83 million AND nodes and 167,333 latches, including the initial-state encoding. Baseline, ISA-constrained, queue-cut, and combined ISA/queue-cut correspondence each reach a one-hour wall-clock limit without an accepted optimized result. Execution-region assumptions, graph prebalancing, and a prebalanced run importing the proved queue invariants also fail to complete within that limit. The plain whole-core controls retain arbitrary initial state without a startup-reset protocol, a more conservative domain than the locally proved reset-reachable cuts. Accordingly, the automatic results above are component-level; the whole-core areas below map the manually generated source cuts.

Physical hardware cost. Separately, we map the generated source cuts directly to identical SKY130 HD typical-corner cells, without formal AIG initial-state encoding. Scheduler/SelectLogic area falls from 21,735.85 to 15 636.25 μm^2, a 28.1% reduction at IQ16. Load/store queue area falls from 86,273.99 to 40 518.86 μm^2, a 53.0% reduction at 16/16 entries. These wrappers retain allocation, status, selection/recovery, and commitment outputs, including raw scheduler pointers; the queue wrapper masks forwarded data and miss status only when forwarding is disabled. These measurements use flip-flop storage and the original, unmasked reset observation boundary. They are component cell areas, distinct from the automatic AIG experiment.

Table 6 gives the matched whole-core mappings. The scheduler-only cut saves 3.36% without requiring the memory-region contract; combining both cuts saves 3.85%. Only the corresponding proved candidate source files differ from the reference; capacities and the FP configuration remain unchanged. These in-process maps include all core logic and flip-flop implementations of caches and register memories. They are not SRAM-macro or post-layout measurements. Component percentages must not be substituted for these whole-core results.

Measured behavior. Full-core Verilator 4.228 simulations of the scheduler cut preserve exact counters, serial output, and four monitored traces for 12 integer diagnostic inputs, CoreMark, Dhrystone, and the three FP-memory inputs. An FP-arithmetic test passes on the original core but violates the contract and fails to complete with the cut. The queue cut is validated separately on stream-sum $(a_i + b_i)$ and stream-mix $((a_i + 3b_i) \text{ xor } \text{rol}_{32}(a_i, 5))$, using 32-bit arithmetic, 512 elements, and three seeds per kernel. All six signatures match independent arithmetic oracles and Spike, and the original cycles and memory/resource traces are unchanged. Two occupied MSHRs remain observed. Repeating the six inputs at backing-read pipeline depths 5, 10, and 20 gives 18 matched original/cut pairs with identical cycles and all five traces, including MSHR and backing-memory events. Request-service parameters, jitter, caches, and capacities are fixed. Combining both cuts also preserves all counters and four monitored traces for the six streaming inputs under both contracts.

Table 7: Full-core application cycle ratios relative to ROB64, IQ16, LQ/SQ16, and INT/FP physical registers 64/64. NC denotes noncompletion within the 20-million-cycle limit.

Changed capacity	CoreMark	Dhrystone
LQ/SQ 8/8	1.016	1.117
LQ/SQ 4/4	1.504	2.249
ROB 32	NC	1.001
ROB 16	1.003	1.108
IQ 32	1.116	1.059
IQ 64	1.629	1.278
INT/FP registers 48/48*	1.002	1.034
INT/FP registers 128/128	0.989	1.002

*Requires an explicit reset-address guard, also applied to the matched 64/64 control, whose measured cycles and traces equal the original.

Capacity changes are a different experiment. Table 7 reports benchmark-loop cycle ratios for CoreMark's 2 KB validation workload (ten iterations) and Dhrystone (500 iterations). Completed runs pass the original checksum/final-state checks; these are cycle measurements, not certified benchmark scores. Each row changes only the named capacity group from the reference. Larger IQs do not improve these examples, and ROB32 fails to complete CoreMark within 20 million simulated cycles, despite passing the shorter diagnostics. Its monitored resource state stops changing; we report noncompletion, not a universal deadlock proof or a fabricated runtime. This failure remains in the dataset.

Implementation legality also matters. IQ4 violates the pinned free-list RAM's bank divisibility, while IQ8 creates one-entry-bank and zero-width index cases. We do not silently change execution ports to enable them. Physical-register counts are coupled between INT and FP. At 48/48, the original reset sweep addresses nonexistent entries, requiring both write suppression and inactive-address clamping; the same guard is identically inactive at 64/64. This is an explicit compatibility change plus resizing, not a proved cycle-equivalent cut. Matched direct mappings retaining all register-file read/write ports reduce that component's area from 1.865 mm^2 at guarded 64/64 to 1.419 mm^2 at guarded 48/48, or 23.9%. This is a component capacity/area tradeoff with the cycle costs in Table 7, not a whole-core Pscorr area reduction.

Limits of architectural and temporal contracts. Pointer chasing returns correct results with the queue cut but produces 2,583–2,711 transient load addresses outside its contract across three inputs; these runs are excluded from positive applicability claims. An intentionally aliasing kernel produces wrong signatures for all three seeds. Architectural nonaliasing alone therefore does not capture the events used by speculative hardware. Likewise, response latency alone does not bound allocation-to-release residency in a ROB, reservation station, physical register, or MSHR. For seed-1 stream-sum, maximum observed backing-read latencies are 22/27/37 cycles at pipeline depths 5/10/20, but complete MSHR valid intervals reach 37/39/50 cycles. These finite maxima are not guaranteed bounds. A certified lifetime bound W and allocation envelope $\alpha(W)$ can bound live entries, with precise clock-edge conventions and the actual admission slack. For example, RSD's load/store queues require occupancy at most $D - R - 1$ before admitting a rename group of width R into depth D. Our capacity sweeps measure tradeoffs; they do not supply that universal lifetime proof.

Application-specific scheduling also arises in dynamically scheduled high-level synthesis (HLS). Recent work replaces generic load/store queues (LSQs) with circuits enforcing only necessary dependencies.[3] Our case study examines the corresponding opportunity across an existing speculative processor's scheduler, memory ordering, and resource capacities, making its execution and observation contracts explicit.

4.2 Generality and Scalability

To assess generality beyond Ibex, we apply the same ISA-constrained pruning flow to a five-stage, in-order Rocket core [2]. We specialize it for 21 workload instruction sets, using a full-RV32IM reference and the same synthesis and mapping flow.

Figure 4 shows a 10.1% geometric-mean mapped-area reduction, with savings of up to 25.9%. Sequential sweeping completes in under four minutes for every workload, with a median of 103 seconds. The Rocket results thus complement the Ibex constraint studies: the shared flow retains useful area savings and practical analysis cost on a different, deeper processor pipeline.

Discussion. The Ibex results demonstrate PDAG-SFA's effectiveness across the evaluated optimization scenarios. The framework successfully identifies and removes unused hardware at multiple abstraction levels, from architectural registers to individual datapath bits. The consistent FMax improvements accompanying area reductions demonstrate that removing unused logic not only saves silicon area but also simplifies critical paths. Most significantly, the data-width optimizations showcase PDAG-SFA's microarchitecture-aware capabilities, achieving substantial savings impossible with ISA-only approaches. These results, obtained in 2 to 4 orders of magnitude less time than exhaustive formal verification,

[3]Pirayadi et al., *Out with LSQs: Custom Circuits for Memory Access Reordering in Dynamic HLS*, FPGA 2026, doi:10.1145/3748173.3779204.

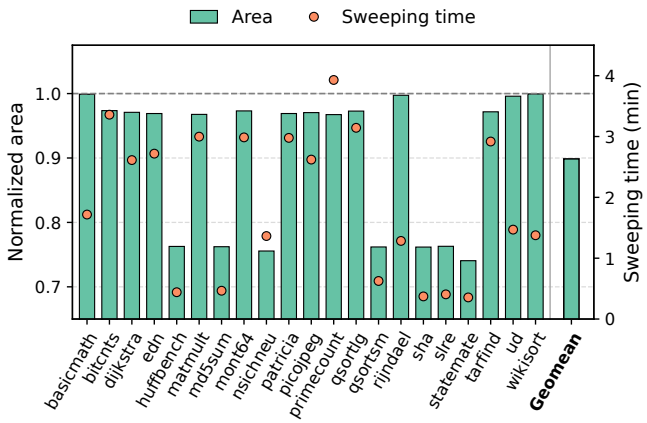


Figure 4: Rocket specialization across 21 workloads: area relative to RV32IM (bars) and sweeping time (dots). Geomean summarizes area.

confirm that semiformal analysis provides a practical path to aggressive hardware specialization without sacrificing correctness guarantees for the constrained usage scenarios.

5 CONCLUSION

This work presented PDAG-SFA, a scalable hardware pruning framework that addresses the fundamental challenges limiting practical ASIP deployment. By embracing semiformal verification techniques and microarchitecture awareness, we overcome the computational intractability and optimization limitations of prior approaches. Our key contributions include: (1) a semiformal verification methodology that trades completeness for practical scalability, enabling 2 to 4 orders of magnitude faster analysis than exhaustive formal methods; (2) a microarchitecture-aware optimization approach that translates diverse constraints including ISA-level, data-level, and interface-level specifications into exploitable don't-care conditions; and (3) the CRISP, a domain-specific language with formal denotational semantics for precise constraint specification.

Experimental evaluation on the Ibex RISC-V core demonstrates the framework's effectiveness across multiple optimization dimensions. Register file optimizations achieve up to 47.3 % area reduction with 80.1 % FMax improvement. ISA subsetting yields 34.7 % area savings for arithmetic-only cores. Most significantly, data-width constraints that are impossible to express in ISA-only frameworks enable 47.6 % area reduction for 8-bit applications. These results, synthesized using the SkyWater 130 nm open-source PDK with ABC technology mapping and OpenSTA timing analysis, validate our approach's practical applicability.

The RSD case study extends the investigation to speculative execution. Local invariants guide automatic component correspondence and justify cuts in scheduler classification and memory-dependence logic at unchanged capacity. Experiments on the ROB, issue queue, and physical registers

separately expose performance tradeoffs and configuration failures. The results identify opportunities across an out-of-order core while keeping execution contracts, observation boundaries, and proof scope explicit.

The implications extend beyond immediate area and performance gains. PDAG-SFA's open-source implementation democratizes advanced ASIP design, particularly crucial for emerging domains like flexible and printed electronics where EDA licensing costs can exceed fabrication expenses. The Rocket study further demonstrates that the shared ISA-constrained pruning flow extends to a deeper pipeline, preserving useful area savings with practical analysis runtime. As embedded systems proliferate and transistor budgets tighten, PDAG-SFA provides a practical path toward ultra-efficient, application-specific processors that extract maximum value from every transistor.

REFERENCES

[1] Luca Amarú, Pierre-Emmanuel Gaillardon, and Giovanni De Micheli. 2015. The EPFL combinational benchmark suite. *Hypotenuse* 256, 128 (2015), 214335.

[2] Krste Asanović, Rimas Avizienis, Jonathan Bachrach, Scott Beamer, David Biancolin, Christopher Celio, Henry Cook, Daniel Dabbelt, John Hauser, Adam Izraelevitz, Sagar Karandikar, Ben Keller, Donggyu Kim, John Koenig, Yunsup Lee, Eric Love, Martin Maas, Albert Magyar, Howard Mao, Miquel Moreto, Albert Ou, David A. Patterson, Brian Richards, Colin Schmidt, Stephen Twigg, Huy Vo, and Andrew Waterman. 2016. *The Rocket Chip Generator*. Technical Report UCB/EECS-2016-17. EECS Department, University of California, Berkeley. https://www2.eecs.berkeley.edu/Pubs/TechRpts/2016/EECS-2016-17.html

[3] Nathaniel Bleier, Calvin Lee, Francisco Rodriguez, Antony Sou, Scott White, and Rakesh Kumar. 2022. FlexiCores: low footprint, high yield, field reprogrammable flexible microprocessors. In *Proceedings of the 49th Annual International Symposium on Computer Architecture* (New York, New York) *(ISCA '22)*. Association for Computing Machinery, New York, NY, USA, 831–846. https://doi.org/10.1145/3470496.3527410

[4] Nathan Bleier, John Sartori, and Rakesh Kumar. 2021. Property-driven Automatic Generation of Reduced-ISA Hardware. In *2021 58th ACM/IEEE Design Automation Conference (DAC)*. 349–354. https://doi.org/10.1109/DAC18074.2021.9586090

[5] Robert Brayton and Alan Mishchenko. 2010. ABC: An academic industrial-strength verification tool. In *International Conference on Computer Aided Verification*. Springer, 24–40.

[6] Hari Cherupalli, Henry Duwe, Weidong Ye, Rakesh Kumar, and John Sartori. 2017. Bespoke Processors for Applications with Ultra-low Area and Power Constraints. In *Proceedings of the 44th Annual International Symposium on Computer Architecture* (Toronto, ON, Canada) *(ISCA '17)*. Association for Computing Machinery, New York, NY, USA, 41–54. https://doi.org/10.1145/3079856.3080247

[7] CHIPS Alliance. 2024. Synlig: SystemVerilog Synthesis Tool. https://github.com/chipsalliance/synlig.

[8] Krishanu Debnath, Rajeev Murgai, Mayank Jain, and Janet Olson. 2018. SAT-based redundancy removal. In *2018 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. 315–318. https://doi.org/10.23919/DATE.2018.8342028

[9] Sushant Dinesh, Yongye Zhu, and Christopher W. Fletcher. 2025. H-Houdini: Scalable Invariant Learning. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1* (Rotterdam, Netherlands) *(ASPLOS '25)*. Association for Computing Machinery, New York, NY, USA, 603–618. https://doi.org/10.1145/3669940.3707263

[10] Daniel Große, Lucas Klemmer, and Dominik Bonora. 2024. Using Formal Verification Methods for Optimization of Circuits Under External Constraints. In *2024 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. 1–6. https://doi.org/10.23919/DATE58400.2024.10546556

[11] Nate Lawson. 2009. Side-channel attacks on cryptographic software. *IEEE Security & Privacy* 7, 6 (2009), 65–68.

[12] Dewmini Sudara Marakkalage, Eleonora Testa, Walter Lau Neto, Alan Mishchenko, Giovanni De Micheli, and Luca Amaru. 2024. Scalable Sequential Optimization Under Observability Don't Cares. In *2024 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. IEEE, 1–6.

[13] Gabriel Cadilha Marques, Farhan Rasheed, Jasmin Aghassi-Hagmann, and Mehdi B. Tahoori. 2018. From silicon to printed electronics: A coherent modeling and design flow approach based on printed electrolyte gated FETs. In *2018 23rd Asia and South Pacific Design Automation Conference (ASP-DAC)*. 658–663. https://doi.org/10.1109/ASPDAC.2018.8297397

[14] Alan Mishchenko, Michael Case, Robert Brayton, and Stephen Jang. 2008. Scalable and scalably-verifiable sequential synthesis. In *2008 IEEE/ACM International Conference on Computer-Aided Design*. 234–241. https://doi.org/10.1109/ICCAD.2008.4681580

[15] Alireza Raisiardali, Konstantinos Iordanou, Jedrzej Kufel, Kowshik Gudimetla, Kris Myny, and Emre Ozer. 2025. Flexing RISC-V Instruction Subset Processors to Extreme Edge. In *Proceedings of the 58th IEEE/ACM International Symposium on Microarchitecture (MICRO '25)*. Association for Computing Machinery, New York, NY, USA, 1147–1159. https://doi.org/10.1145/3725843.3756036

[16] Pasquale Davide Schiavone, Francesco Conti, Davide Rossi, Michael Gautschi, Antonio Pullini, Eric Flamand, and Luca Benini. 2017. Slow and steady wins the race? A comparison of ultra-low-power RISC-V cores for Internet-of-Things applications. In *2017 27th International Symposium on Power and Timing Modeling, Optimization and Simulation (PATMOS)*. IEEE, 1–8.

[17] Subhash Sethumurugan, Shashank Hegde, Hari Cherupalli, and John Sartori. 2022. A scalable symbolic simulation tool for low power embedded systems. In *Proceedings of the 59th ACM/IEEE Design Automation Conference* (San Francisco, California) *(DAC '22)*. Association for Computing Machinery, New York, NY, USA, 175–180. https://doi.org/10.1145/3489517.3530433

[18] Qilin Si, Prattay Chowdhury, Rohit Sreekumar, and Benjamin Carrion Schafer. 2023. Application Specific Approximate Behavioral Processor. *IEEE Transactions on Sustainable Computing* 8, 2 (2023), 165–179. https://doi.org/10.1109/TSUSC.2022.3222117

[19] Stian Gerlach Sørensen, Christian Bartsch, Dominik Stoffel, and Wolfgang Kunz. 2022. Generation of Formal CPU Profiles for Embedded Systems. In *2022 IFIP/IEEE 30th International Conference on Very Large Scale Integration (VLSI-SoC)*. 1–6. https://doi.org/10.1109/VLSI-SoC54400.2022.9939572

[20] Qinhan Tan, Aarti Gupta, and Sharad Malik. 2022. Usage-Based RTL Subsetting for Hardware Accelerators. In *2022 IEEE/ACM International Conference On Computer Aided Design (ICCAD)*. 1–9.